ARTIFICIAL INTELLIGENCE

PROJECT REPORT

Map Coloring Using Constraint Satisfaction Problem

SUBMITTED BY

Sana Goel 245811436	**Riti Grover** 245811426
Shruti Gupta 245811522	**Pulagam Sajani** 245811352

Table of Contents

1	Introduction	1
2	Problem Definition	2
3	Algorithm Explanation (Backtracking)	3
4	Implementation Details	4
5	Code Snippet	5
6	Results & Output	6
7	Performance Analysis	7
8	Comparison with Forward Checking	9
9	Use Cases of CSP	10
10	Conclusion	11

1. Introduction

Constraint Satisfaction Problem (CSP) is an important concept in Artificial Intelligence where a solution must satisfy a set of constraints. A CSP consists of three main components:

Variables	Elements that need to be assigned values
Domains	Possible values each variable can take
Constraints	Rules that restrict the assignment of values

The objective is to assign values to all variables such that all constraints are satisfied.

CSPs are widely used in:

- Scheduling problems
- Puzzle solving (e.g., Sudoku, N-Queens)
- Resource allocation

2. Problem Definition

The Map Coloring Problem is a classic CSP problem.

Objective

Assign colors to different regions of a map such that no two adjacent regions share the same color.

Constraint

No two adjacent regions should have the same color.

Representation

Component	Representation
Each region	Variable
Available colors	Domain
Adjacency between regions	Constraints

In this project, the Australia map is used with the following seven regions:

> **Regions:** WA, NT, SA, Q, NSW, V, T

3. Algorithm Explanation — Backtracking

Backtracking is a recursive search algorithm used to solve Constraint Satisfaction Problems. It works by trying possible solutions and backtracking whenever a constraint is violated.

Working Principle

The algorithm systematically explores the search space by assigning values to variables one at a time, and reverting (backtracking) whenever a constraint violation is detected.

Step-by-Step Process

1	Select an unassigned region
2	Assign a color from the available colors
3	Check if the assignment satisfies all constraints
4	If valid → proceed to the next region
5	If invalid → backtrack and try another color
6	Repeat until a solution is found or all possibilities are exhausted

4. Implementation Details

Approach Used

Backtracking Algorithm

Data Structures

Data Structure	Purpose
Dictionary	Represents the graph as an adjacency list
List	Stores the available colors
Dictionary (result)	Stores the final color assignments

Key Functions

is_valid(region, color): Checks whether assigning a color to a region violates constraints. Ensures no neighbor has the same color.

backtrack(): Recursive function that assigns colors and explores all possibilities, backtracking when necessary.

5. Code Snippet

The following Python implementation demonstrates the backtracking algorithm for the Australia Map Coloring CSP:

```
graph = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q':  ['NT', 'SA', 'NSW'],
    'NSW':['Q',  'SA', 'V'],
    'V':  ['SA', 'NSW'],
    'T':  []
}

colors = ['Red', 'Green', 'Blue']
result = {}

def is_valid(region, color):
    for neighbor in graph[region]:
        if neighbor in result and result[neighbor] == color:
            return False
    return True

def backtrack():
    if len(result) == len(graph):
        return True
    for region in graph:
        if region not in result:
            for color in colors:
                if is_valid(region, color):
                    result[region] = color
                    if backtrack():
                        return True
                    del result[region]
            return False

if backtrack():
    print("Solution Found:")
    for region in result:
        print(region, "->", result[region])
else:
    print("No solution exists")
```

6. Results & Output

The algorithm successfully assigns colors such that all constraints are satisfied.

Sample Output

```
Solution Found:
WA      ->  Red
NT      ->  Green
SA      ->  Blue
Q       ->  Red
NSW     ->  Green
V       ->  Red
T       ->  Red
```

Observations

- No adjacent regions share the same color
- Valid CSP solution obtained using only 3 colors

7. Performance Analysis

Time Complexity

Worst-Case Time Complexity: O(d^n) where d = number of colors, n = number of regions

For each region, the algorithm tries all possible colors. In the worst case, it explores all combinations, resulting in exponential growth as the input size increases.

Input Size	Performance Impact
3 regions	Very fast — minimal backtracking
10 regions	Significantly slower — more combinations
Large n	Exponential slowdown — impractical without pruning

Space Complexity

Space Complexity: O(n) — stores one color assignment per region, plus a recursion stack of depth n

Completeness

Backtracking is a complete algorithm — it explores all possible combinations and is guaranteed to find a solution if one exists.

Optimality

Backtracking is not always optimal. It finds a valid solution but does not necessarily minimize the number of colors used. The algorithm may use more colors than theoretically required.

8. Comparison with Forward Checking

Feature	Backtracking	Forward Checking
Approach	Try all possibilities	Prunes invalid options early
Speed	Slower	Faster
Efficiency	Low for large problems	Higher efficiency
Search Space	Full exploration	Reduced

Advantages of Backtracking

- Simple to implement
- Works for all CSP problems
- Guarantees a solution (complete)

Limitations of Backtracking

- High time complexity
- Slow for large inputs
- Explores unnecessary paths

Advantages of Forward Checking

- Reduces search space early
- Faster than pure backtracking
- More efficient for large CSPs

9. Use Cases of CSP

Map coloring — assign colors to geographic regions
Sudoku solving — fill in a grid satisfying row/column/box constraints
Timetable scheduling — assign time slots without conflicts
Resource allocation — distribute resources meeting capacity constraints
AI planning problems — sequence actions satisfying preconditions

10. Conclusion

Map Coloring is a fundamental Constraint Satisfaction Problem in Artificial Intelligence. This project demonstrates how the Backtracking algorithm can be used to solve CSPs effectively.

Key Takeaways

1. Backtracking ensures correctness but is computationally expensive for large inputs.
2. Performance can be improved using Forward Checking, which prunes the search space early.
3. Further optimization is possible through heuristics such as MRV (Minimum Remaining Values) and the Degree Heuristic.
4. This project highlights how AI constraint-solving techniques can address real-world constraint-based problems efficiently.

Overall, this project provides a clear and systematic exploration of how AI techniques can be applied to solve constraint-based problems, with practical insights into performance trade-offs and areas for future improvement.